AI ENGINEERING | CHAPTER 8

Dataset Engineering

Detailed study notes from the lecture transcript

A model's capability depends on both its architecture and the data it learns from. Dataset engineering builds the most useful training set possible within quality, coverage, privacy, and budget constraints.

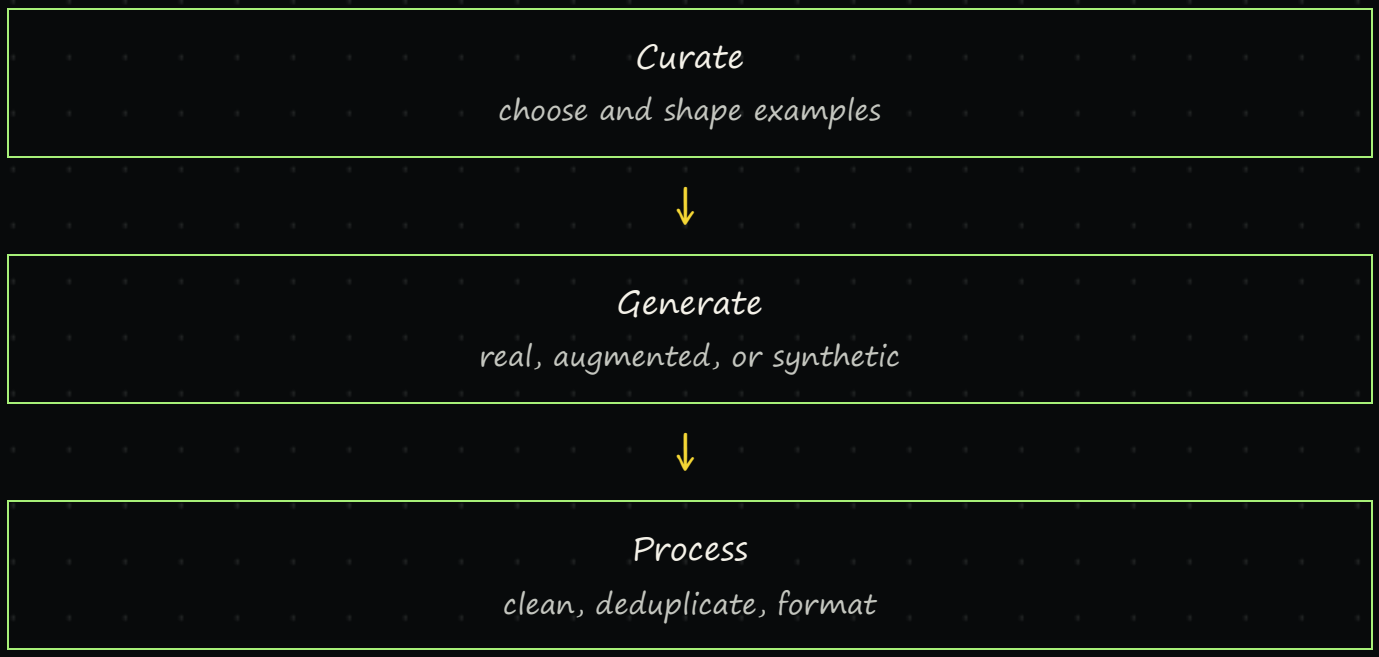
01 Why Dataset Engineering?

The data-centric view

Model-centric AI seeks progress mainly through better architectures and models. **Data-centric AI** seeks progress through better data, curation, and processing methods. Dataset engineering emphasizes the latter.

As many teams build services on top of existing open-source or foundation models, high-quality task data becomes an important differentiator. The goal is not simply to collect more data; it is to create data that teaches the desired capability efficiently.

Dataset engineering loop



Repeat the loop using evaluation results; keep the task and budget in view.

What is counted?

- **Pre-training:** commonly discussed in number of tokens.
- **Post-training:** commonly discussed in examples, such as instruction–response pairs or preference comparisons.

The unit reflects the training stage and its data format.

Three training stages

Pre-training gives broad patterns and knowledge; **supervised fine-tuning (SFT)** teaches task behavior from demonstrations; **preference fine-tuning** aligns outputs using comparative preferences or related feedback.

The same data proportions are not automatically right for every stage.

02 Curation: Teach the Intended Behavior

Match the data shape to the learning objective

Training objective	Typical example structure
Self-supervised learning	A sequence of tokens or data for the model to continue/predict.
Instruction fine-tuning	Instruction or prompt + target response.
Preference fine-tuning	Instruction + preferred (winning) response + dispreferred (losing) response.
Reward-model training	Instruction + response + preference score or label.

Reasoning and tool use

- **Chain-of-thought demonstrations:** worked examples show intermediate reasoning steps. For a multi-step arithmetic question, demonstrate each operation rather than only the final number.
- **Tool-use examples:** show when and how a tool is used, including the resulting observation and response.
- **Single-turn:** one instruction followed by one answer.
- **Multi-turn:** the assistant can clarify intent, ask follow-up questions, use tools, and continue the exchange.

Training examples should reflect the interaction pattern expected at inference time.

What makes data high quality?

- **Relevant:** appropriate to the model's real use case and domain.
- **Task-aligned:** follows the intended instructions, audience, tone, and safety requirements.
- **Consistent:** examples use compatible labeling and response standards.
- **Correctly formatted:** matches the schema, chat template, or tool-call format the model must learn.
- **Unique:** avoids unnecessary duplication and evaluation leakage.
- **Compliant:** respects privacy, licensing, regulation, and applicable norms.

Quality can outweigh raw count

The lecture describes the LIMA result as an example: a 65-billion-parameter LLaMA model fine-tuned on about 1,000 carefully curated instructions reportedly performed better than GPT-4 in 43% of that study's comparisons.

This illustrates the value of careful examples; it is **not** a general claim that 1,000 examples always outperform larger datasets or stronger models. Human-created data can contain errors, so review and AI-assisted annotation may help, with quality checks.

Coverage and diversity

Coverage means representing the range of problems the model is expected to solve. **Diversity** means varying the topics, inputs, language, task types, edge cases, and speaking patterns within that range.

- Include the chatbot's expected subject areas.
- Represent local dialects and different ways people phrase requests.
- Include relevant math and high-quality code tasks.
- Cover common cases and important rare or difficult cases.
- Check for missing categories, not only total example count.
- Keep examples aligned with current real-world topics and use.

Small, high-quality math and code fine-tuning sets have been reported to improve broader model benchmarks; structured data can contribute useful, diverse patterns.

03 Data Mix, Quantity, and Fine-Tuning Choices

Choose a stage-specific mixture

A data mixture might combine general knowledge, question answering, math, code, and other task data. The useful proportions can differ between pre-training, SFT, and preference fine-tuning.

Do not assume the pre-training mix should simply be copied into every later stage. Experiment with proportions and measure the capabilities that matter.

> Practical approach: test candidate mixtures on smaller, faster-to-train models, then use scaling-law evidence to inform larger runs. Treat extrapolation as an estimate that still needs validation.

Quantity is important, but not alone

The lecture gives approximate scale examples: LLaMA 2 trained on around **2 trillion tokens** and LLaMA 3 on around **16 trillion tokens**. Large pre-training runs need enormous token volumes, but post-training value also depends on example quality and task coverage.

Diminishing returns: adding examples often improves performance, but the gain per added example tends to shrink. The first well-designed examples can have especially high value; later additions should target remaining gaps.

Three factors for fine-tuning decisions

1. **Data amount:** how many trustworthy, relevant examples are available?
2. **Task complexity:** how much behavior must the model learn?
3. **Base-model capability:** how well does the starting model already perform?

Full fine-tuning updates the model broadly and generally needs more data and resources. **PEFT** (parameter-efficient fine-tuning) updates a smaller set of parameters, making it useful when data or compute is limited.

Rule of thumb from the lecture

- **Few examples:** prefer a capable base model and PEFT; stronger models can often learn more from limited demonstrations.
- **Many examples:** consider full fine-tuning of a smaller, less expensive model when evaluation shows comparable task performance.

The lecture cites an SNLI experiment: the gap between weaker and stronger models was more pronounced with very little fine-tuning data and narrowed as the number of examples grew. These are decision heuristics, not universal laws.

Start small; let evaluation decide whether to scale



If a small set produces gains, add targeted examples. If it does not, diagnose the task definition, labels, formatting, and example quality before scaling. A staged approach can also work: broad or less task-specific learning first, then a smaller high-quality task-specific set.

Fine-tuning across a **diverse set of tasks** can improve held-out performance and often helps generalization, with gains eventually plateauing. Measure both task breadth and model scale.

Fine-tuning may not always change a model as expected

The lecture refers to **model ossification**: in some cases, fine-tuning (particularly on small models or with limited adaptation) may not shift parameters or behavior enough to achieve the desired change. Treat this as a warning to evaluate the actual before-and-after behavior and consider the method, data, and base model together.

---

04 Acquisition, Annotation, and Augmentation

Acquire data responsibly

- Respect user privacy and applicable data regulations.
- Confirm that source data and any model-generated derivatives may be used for the intended purpose.
- Track provenance and rights; synthetic output does not automatically remove restrictions inherited from its source or generator.

Data flywheel: people use a product, their interactions or feedback produce useful signals, and approved signals can help improve the product. User-provided content and system-generated usage/feedback data are distinct sources and need appropriate governance.

Annotation guidelines

Write clear, specific instructions for labelers and reviewers. Define the schema, edge cases, what counts as a correct answer, and how to handle ambiguity. Consistent rules reduce disagreement and make the dataset teach a coherent behavior.

Augmentation vs. synthesis

Method	Core idea	Example
Data augmentation	Transform or perturb existing real examples while preserving the intended label or meaning.	Crop, flip, or rescale an image; carefully perturb text or pixels.
Data synthesis	Create new examples that mimic important properties of real data.	Generate a new scenario, instruction, or simulated interaction.

Augmentation can improve robustness to variation and help address representation gaps, but transformations must preserve correctness. A perturbed image should still depict the intended class.

Why synthesize data?

- **Quantity:** produce more examples at useful scale.
- **Coverage:** fill sparse areas, rare cases, and adversarial conditions.
- **Quality:** create preference or task examples with consistent structure, then verify them.
- **Privacy:** support development where sensitive real records cannot be shared or reused.
- **Distillation:** use a larger teacher model's outputs to train a smaller student model.

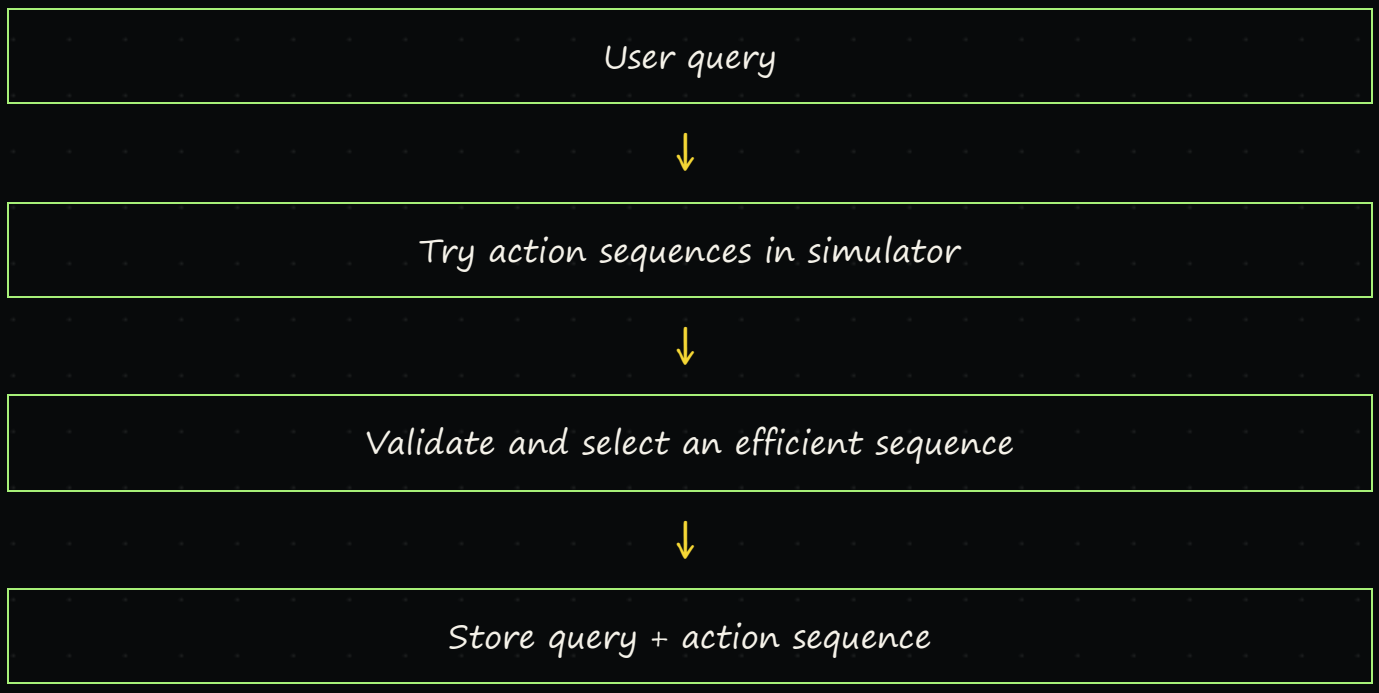
05 How Synthetic Data Is Generated

Procedural generation

Algorithms create examples according to explicit rules or a simulated environment.

- **Rule-based templates:** define fields, then populate them with controlled values or a random-data tool. Example fields could include age, blood group, and symptoms in a fictional health record.
- **Transformations:** perturb data, replace selected tokens, or modify pixels to build robustness examples. Labels must remain valid.
- **Simulation:** execute possible actions in a virtual environment, test outcomes, and record successful trajectories. This can be cheaper and safer than collecting every event in the physical world.

Simulation for tool use



Simulators can create examples for rare or costly events and for tool interactions such as navigating an interface. The action trajectory should be checked before being treated as a target demonstration.

AI-powered synthesis

- Use a model to paraphrase, translate, or vary seed examples.
- Ask a model to simulate API outputs for offline dataset creation when live calls are costly; verify that simulated outputs are realistic.
- Translate high-resource-language data into a lower-resource language, then use back-translation or other review to check meaning preservation.
- Use an AI judge with response order swapped to reduce first-position bias. Prefer a candidate only if its quality remains supported when ordering changes.

Illustrative lecture examples: MetaMath expanded roughly 15,000 math seeds to about 400,000 variants; reported gains depend on the data, model, and evaluation.

Instruction datasets from seed examples

UltraChat: the lecture describes using a model to expand topics into subtopics and generate multi-turn instruction-dialogue examples across a broad topic range.

Alpaca: the lecture describes using about 175 seed instructions and a GPT-3-family model to generate varied instruction-response examples, expanding the collection to roughly 52,000 examples.

Generated examples need screening for correctness, diversity, and repetition; count alone is not a quality guarantee.

Reverse instruction: start from trusted content

Long AI-written answers may hallucinate. A reverse-instruction method starts with existing high-quality content and asks a model to generate an instruction or question that the content answers. This is an inverse mapping: content → plausible prompt.

1. Start with a small seed set and train an initial, weaker model.
2. Provide high-quality source content and ask the model to produce instruction-content pairs.
3. Fine-tune on reviewed pairs, then repeat with suitable source material.
4. Stop when measured target performance is sufficient; retain source rights and provenance.

The content source must be legally and ethically usable. Iteration can improve the generator, but it can also amplify errors if generated pairs are not checked.

---

06 Two Useful Synthesis Workflows

Long-context fine-tuning: 8K to 64K example

When a model answers well from a short context but struggles to find information in a much longer one, use the short-context ability to create reliable answer targets first.

1. Split a long document into chunks that fit the known reliable context size (the lecture example uses 8K-token chunks from a 64K-token document).
2. Create question-answer pairs for each chunk, where the answer is grounded in that chunk.
3. For fine-tuning, present each question with the **full 64K context** and train toward the answer already established from the smaller chunk.

This teaches the model to locate relevant evidence across the longer input while avoiding the need to trust an answer generated directly from a context length it cannot yet handle reliably.

LLaMA 3 code-data synthesis, as described

1. Generate a programming problem description.
2. Generate solutions in multiple programming languages.
3. Generate unit tests and run them to verify solutions.
4. Use failures to identify and repair incorrect solutions.
5. Translate a verified fix across the other languages, then test those versions too.
6. Add code explanations or conversations so the model learns to discuss code as well as produce it.

The lecture reports creating about **2.7 million synthetic code-related examples** for supervised fine-tuning of LLaMA 3.1. The key idea is to combine generation with executable validation, not to accept generated code unchecked.

Verification should match the task

Can correctness be executed or measured directly?

Use tests, constraints, or deterministic checks first.

↓

If not, use expert review or a calibrated AI verifier

Record uncertainty and audit samples.

↓

Keep only examples that meet the quality bar

Code is a convenient synthesis domain because unit tests can check functional behavior. For open-ended text, AI verifiers can score examples or assign topic metadata, but they can share the generator's biases and should not be treated as ground truth.

Back-translation: translate an example from language A to B, then use a trusted translator to translate it back to A. Compare meaning, not only exact wording. It is a useful sampled check for translation quality, not proof that every example is correct.

07 Risks and Limits of AI-Generated Data

1. Quality control is hard

Reliable metrics and review methods are needed. A detector that cannot distinguish a synthetic example from real data does not by itself prove the example is accurate, useful, or safe.

2. Superficial imitation

A student model may copy a teacher's style and frequent patterns without learning factual consistency or generalizing to real inputs. This can preserve or introduce hallucinations.

3. Mode collapse and diversity loss

Generators tend to produce common, high-probability cases more often than rare ones. Training repeatedly on generated outputs can amplify common patterns and erase rare cases over iterations.

> common patterns ↑ each round | rare patterns ↓ each round

Keep real-data anchors, explicitly preserve rare-case coverage, deduplicate, and audit category distributions over successive generations.

4. Bias and lineage

Synthetic data may reproduce the generator's biases. Its provenance can also become difficult to trace, including whether the generator learned from material that cannot legally or ethically be reused.

Track source, model, prompt, generation date, review status, and permissions where feasible.

Think in terms of risk controls, not “synthetic = safe”

- Review representative samples manually.
- Use executable checks wherever possible.
- Measure coverage and rare-case retention.
- Compare generated examples with trusted real examples.
- Track lineage, licensing, privacy, and consent.
- Evaluate the trained model on held-out real data.

---

08 Preprocessing: Clean, Deduplicate, Format

Inspect and clean

Manually inspect representative records and remove irrelevant artifacts such as unwanted HTML tags, malformed markup, or noise. Filtering should preserve useful meaning and not silently remove important subgroups.

Why deduplication matters

Repeated training examples can over-weight a narrow pattern; duplicates between training and evaluation can contaminate test results and make reported performance misleading. Deduplicate before training and check for train-test overlap.

The lecture emphasizes that even a small duplicated fraction, repeated many times, can materially affect measured performance.

Ways to find duplicates

- **Pairwise comparison:** exact match, n-gram overlap, or semantic similarity. Direct all-pairs comparison is expensive at scale.
- **Hashing/bucketing:** group likely matches, then compare only within candidate buckets to reduce work.
- **Embedding reduction:** reduce vector dimensionality before similarity comparisons to lower computation; preserve enough information for the similarity task.

Clean and filter deliberately

- **Active learning:** prioritize examples expected to be most informative for the model or labelers.
- **Importance sampling:** sample strategically rather than uniformly when the task and sampling weights justify it.
- Track filtering rules and inspect what gets removed so rare but important cases are not lost.

Format examples to match the chat template

The model learns structure as well as meaning. Use the same role markers, separators, field names, and answer boundaries at training and inference time.

Consistent examples

Input: Item: burger
Target: **edible**

Input: Item: car
Target: **inedible**

Input: Item: mushroom
Target: **edible**

Keep the same pattern at use time

Input: Item: vegetable
Expected target: the label in the trained format.

If training examples use a precise prefix or delimiter, changing it at inference can change the model's behavior. Treat the template as part of the interface contract.

Chapter takeaway

Define the behavior first; curate examples that teach it; measure quality, coverage, and the data mix; acquire and synthesize responsibly; verify generated examples; then clean, deduplicate, and format consistently. Scale only when evaluation shows what additional data should accomplish.



Next topic mentioned in the lecture: inference optimization, with attention to serving cost and latency.

Prepared from the supplied Chapter 8 transcript. Examples and reported figures are attributed to the lecture and should be interpreted in their experimental context.